

1. Processes

When a program is loaded into memory and execution begins, the operating system creates a process to run it, therefore a process is an executing program together with its allocated address space, registers, stack, heap, and other operating-system resources. Each process has its own *address space* and usually one *thread of control*. Multiple processes can execute the same program but each of them has its own copy of the program within its own address space and the execution is independent.

Processes are organized hierarchically: each process has a *parent* process which created it, the processes created by a given *parent* are called its *child* processes. Child processes inherit many of their attributes from the parent process.

A process

- holds the program's code and data in memory in its *virtual address space*
- provides an execution context (CPU registers, stack, heap)
- owns OS resources (file descriptors, signals, etc..)

Let's look at the following command

```
[linux@process] sleep 100 &  
[1] 195691
```

The `sleep` process is running and the kernel has assigned the PID (*process identifier*) 195691

Let's look at which process has created the new process `sleep`

```
[linux@process] ps -ef | grep sleep  
process  195691  6159  0 21:00 pts/0  00:00:00 sleep 100  
  
[linux@process] ps -ef | grep 6159  
process  6159  6151  0 Jan07 pts/0  00:00:03 zsh
```

It is the process 6159 that is the `zsh` shell.

The `sleep` process is a *child* of the `zsh` shell that is its *parent* process. The PPID (*parent process identifier*) of `sleep` is 6159.

Every process has a unique process ID that is a non-negative integer. Since the PID is always unique, some applications use it as part of a filename to generate unique filenames. When a process terminates, its PID becomes available and can be reused.

Let's look at the following command

```
[linux@process] ps -ef | more  
UID      PID  PPID  C  STIME TTY      TIME CMD  
root      1    0    0 Jan07  ?       00:00:05 /sbin/init splash  
root      2    0    0 Jan07  ?       00:00:00 [kthreadd]  
...output...
```

`ps`¹ lists the running processes. We can see from the output the two processes with PIDs 1 and 2.

The `init` process is the first *user-space process* started by the system and is the ancestor of all other user-space processes. `init` is invoked by the kernel at the end of the bootstrap procedure. The *kernel* has created it during the boot and it has always the PID 1. On modern Linux systems, `systemd` is used as an *init system*, on these systems `systemd` runs as PID 1 and fulfills the `init` role. On Linux, whatever user-space program runs as PID 1 is the *init system*.

If we look at the `/sbin/init` file we see it is a *symbolic link* to `systemd`

```
[linux@process] ls -l /sbin/init
lrwxrwxrwx 1 root root 22 Nov 18 01:55 /sbin/init -> ../lib/systemd/systemd
```

The second line of the `ps` output lists `kthreadd` that is the *kernel thread manager*. `kthreadd` is responsible for creating and managing other kernel threads. When the *kernel* needs a new kernel thread, it asks `kthreadd` to create it.

Programs in the square brackets are not *user-space processes* but *kernel threads*.

Kernel threads:

- run in kernel space only
- have no user-space address space
- do not execute a program from disk
- do not have a controlling terminal

`init/systemd` (PID 1) is the first user-space process started by the kernel and is responsible for bringing the system to a usable state, while `kthreadd` (PID 2) is a kernel thread daemon responsible for creating and managing other kernel threads in Linux.

Both PID 1 (`init/systemd`) and PID 2 (`kthreadd`) are created by the *kernel* (from the initial *idle task*, PID 0). The *idle task*, historically called the *swapper*, is a special kernel scheduling entity that runs when no other tasks are runnable.

The data type used to represent a process ID is `pid_t` that is defined in `sys/types.h` and other headers². A PID can also be stored in the `id_t` type which can be used to store `uid_t` and `gid_t` types too.

¹ See the `ps(1)` manual page

² See the `id_t(3type)` manual page

1.1 Getting the process identifier: `getpid` and `getppid`

To retrieve the PID of a process we can use the `getpid`³ function that is defined as follows

```
#include <unistd.h>

pid_t getpid(void);

pid_t getppid(void);
```

Return: PID/PPID of the calling process

The `getpid` returns the PID of the calling process.

The `getppid` returns the PPID of the calling process, this will be either the ID of the process that created this process using `fork`, or, if that process has already terminated, the ID of the process to which this process has been reparented (either `init` or a "subreaper" process defined via the `prctl PR_SET_CHILD_SUBREAPER` operation). If the caller's parent is in a different *PID namespace* `getppid` returns 0.

These two functions are always successful.

The `getpid.c` program retrieves and prints the PID and PPID of the calling process

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process ID: %d", (long)getpid());
    printf("\nParent process ID: %d", (long)getppid());
    return 0;
}
```

`getpid.c`

We execute

```
[linux@process] ./getpid
Process ID: 224570
Parent process ID: 223625
```

A process has also a real and effective user and group ID. We have already seen the related functions in the chapter dedicated to users and groups.

3 See the `getpid(2)` manual page

1.2 Creating a process: fork

A running process can create a new process via the `fork`⁴ function.

```
#include <unistd.h>

pid_t fork(void);

Returns: PID of the child in the parent, 0 in the child, on error -1 in the
parent and errno is set
```

`fork` creates a new process by duplicating the calling process, the new process is referred to as the *child* process, while the calling process is referred to as the *parent* process. The *child* process and the *parent* process run in separate memory space.

After a successful call, the PID of the child process is returned to the parent and 0 is returned in the child. If the call fails no child process is created and -1 is returned to the parent.

The `forkpid.c` program creates a new process via `fork`.

```
1 #include <stdio.h>
2 #include <unistd.h>

3 int main(void)
4 {
5     printf("PID: %ld\n", (long)getpid());
6     printf("PPID: %ld\n\n", (long)getppid());
7     fflush(stdout);

8     pid_t pid = fork();
9     if (pid > 0) {
10         printf("Parent: PID=%ld\n", (long)getpid());
11         sleep(2);
12     } else if (pid == 0) {
13         printf("Child: PID=%ld\n", (long)getpid());
14         printf("Child PPID: %ld\n", (long)getppid());
15     } else {
16         perror("fork");
17     }
18     return 0;
19 }
```

forkpid.c

After printing the PID and PPID of the process we create a new process at line 8 and store the PID in the `pid` variable. Next, we check if we are in the parent process (`pid > 0`) and print the parent PID (10). That must be the same as the process PID. We wait for 2 seconds in the parent, check if we are in the child (`pid == 0`) and print the PID of the child and the PPID of the child. If `fork` has returned another value we print the error at lines 15-16.

4 See the `fork(2)` manual page

We execute the program.

```
[linux@process] ./forkpid
PID: 457289
PPID: 223625

Parent: PID=457289
Child: PID=457290
Child PPID: 457289
```

As we can see, the PID of the parent is the same as the PID of the main program (457289), the child has a new PID (457290) and the PPID of the child is the PID of the parent.

We comment out the line 11 and execute.

```
[linux@process] ./forkpid
PID: 458463
PPID: 223625

Parent: PID=458463
Child: PID=458464
Child PPID: 2865
```

We observe that the PPID of the child is not equal to the PID of the parent because the parent process terminates before the child executes `getppid`. As a result, the child becomes an *orphan* and is re-parented to another process. `getppid` returns the current parent, not the original one.

We check what process is the new parent.

```
[linux@process] ps -ef | grep 2865
process    2865      1 0 Jan07 ?        00:00:01 /usr/lib/systemd/systemd -user
...output...
```

A process that becomes an *orphan* is typically re-parented to `init/systemd` (PID = 1).

The `fork.c` program is taken from the `fork(2)` manual page

```
1 #include <signal.h>
2 #include <stdint.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>

6 int main(void)
7 {
8     pid_t pid;

9     if (signal(SIGCHLD, SIG_IGN) == SIG_ERR) {
10         perror("signal");
11         exit(EXIT_FAILURE);
```

```
12     }

13     pid = fork();
14     switch (pid) {
15         case -1:
16             perror("fork");
17             exit(EXIT_FAILURE);
18         case 0:
19             puts("Child exiting.");
20             fflush(stdout);
21             _exit(EXIT_SUCCESS);
22         default:
23             printf("Child is PID %jd\n", (intmax_t) pid);
24             puts("Parent exiting.");
25             exit(EXIT_SUCCESS);
26     }
27 }
```

fork.c

The program sets the `SIGCHLD` signal to be ignored via the `signal` function at line 9 and then it invokes `fork` to create the new process. In the child (18-21), it prints on `stdout` with `puts`, flushes `stdout` and terminates using `_exit`. It uses `_exit` at the place of `exit` to avoid flushing the `stdout` buffer again and to prevent duplicate output. In the parent (23-25), it prints the child PID, a string, and then terminate using `exit`.

We execute the program.

```
[linux@process] ./fork
Child is PID 499364
Parent exiting.
Child exiting.
```

When a child process terminates, the kernel sends a `SIGCHLD` signal to the parent. If this signal is ignored, the terminated child does not become a *zombie*: the kernel automatically reaps it and discards its exit status. In this way, the parent can avoid calling `wait` or `waitpid` to achieve the same result.

A *zombie process* is a process that has terminated execution but still has an entry in the *process table* because its parent has not yet collected its *exit status*. It is a terminated child process that has not yet been reaped by its parent.

When a process becomes a *zombie* all its resources are released except the PID, exit status and some accounting information. It remains in this state until the parent calls `wait` or `waitpid`. A zombie process is listed in the `ps` output with state `Z` or `<defunct>`. *Zombie* exist because the kernel must keep the exit status in a way the parent can retrieve it.

If the parent calls `wait` or `waitpid` the *zombie process* is reaped, if the parent exits the *zombie* is re-parented to `init/systemd` and this will reap it automatically.

Zombie processes do not run, do not use CPU, and do not hold memory or open files; they only retain a PID and minimal process table information until reaped; they only exist so the parent can collect the exit status.

The `zombie.c` program deliberately creates a *zombie process*.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>

4 int main(void)
5 {
6     pid_t pid = fork();
7     if (pid < 0) {
8         perror("fork");
9         exit(EXIT_FAILURE);
10    }
11    if (pid == 0) {
12        printf("Child exiting (PID=%ld)\n", (long)getpid());
13        exit(EXIT_SUCCESS);
14    } else {
15        printf("Parent sleeping (PID=%ld), child PID=%ld\n",
16              (long)getpid(), (long)pid);
17        sleep(30);
18        printf("Parent exiting\n");
19    }
20    return 0;
21 }
```

zombie.c

The `sleep` call at line 16 causes the parent to pause execution without calling `wait` to retrieve the child's exit status. As a result, the child exits immediately and becomes a *zombie*. The zombie remains until the parent process terminates (or calls `wait`).

We execute the program.

Shell 1

```
[linux@process] ./zombie
Parent sleeping (PID=511384), child PID=511385
Child exiting (PID=511385)
```

Shell 2

```
[linux@process] ps -ef | grep zombie
process    511384 223625 0 19:32 pts/2    00:00:00 ./zombie
process    511385 511384 0 19:32 pts/2    00:00:00 [zombie] <defunct>
```

As the output shows, since we have not called `wait` in the parent, the child is *zombie* process. If we replace the `sleep` call with

```
wait(NULL);
```

the parent waits for the child and no zombie is created. The `wait` call blocks the execution until the child terminates.

To prevent zombies, the parent must call `wait` or `waitpid` before or instead of sleeping or exiting.

Variables behavior

The `forkvar.c` program⁵ shows how changes to variables in the child do not affect the value of the variables in the parent.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>

4 int globvar = 6;
5 char buf[] = "a write to stdout\n";

6 int main(void)
7 {
8     int var;
9     pid_t pid;
10    var = 88;

11    if (write(STDOUT_FILENO, buf, sizeof(buf)-1) != sizeof(buf)-1)
12        perror("write");

13    printf("before fork\n");
14    if ((pid = fork()) < 0) {
15        perror("fork");
16    } else if (pid == 0) {
17        globvar++;
18        var++;
19    } else {
20        sleep(2);
21    }

22    printf("pid = %ld, glob = %d, var = %d\n", (long)getpid(), globvar,
23        var);
24    exit(0);
25 }
```

forkvar.c

The lines 4-5 define *external variables* that are stored in the *initialized data section*. On the other hand, at lines 8-10, the program defines *automatic variables* that are stored in the *stack*. The line 11 prints the `buf` contents on `stdout` and the line 14 create a new process using `fork`. The child

⁵ The program is taken from the *Advanced Programming in the UNIX Environment* book

process (16-18) modifies the `globvar` and `var` values, while the parent sleeps. At line 22, the program prints the PID and the variables values.

We execute the program.

```
[linux@process] ./forkvar
a write to stdout
before fork
pid = 523361, glob = 7, var = 89
pid = 523360, glob = 6, var = 88
```

The first line after the “before fork” statement, is the child output with the incremented values; the second line is the parent output with the original values.

We take a look at what happens if we redirect the output to a file.

```
[linux@process] ./forkvar > fork.log

[linux@process] cat fork.log
a write to stdout
before fork
pid = 528754, glob = 7, var = 89
before fork
pid = 528753, glob = 6, var = 88
```

The `write` function is not buffered and is called before the `fork`, therefore its data is written once to `stdout`.

The *standard I/O library* is buffered and the `stdout` is *line buffered* if connected to a terminal and *fully buffered* if connected to a file. In the first execution the `printf` writes on `stdout` that is flushed by the newline (*line buffered*), in the second execution the line remains in the buffer (`stdout` is *fully buffered*) when `fork` is called and the buffer is copied into the child. As a result `printf` is called once but the file contains two “before fork” write.

The second `printf` just appends the data to the existing buffer.

Parent and child features

A child process is an exact duplicate of the parent process except for the following points⁶

- The child has its own unique *process ID*, and this PID does not match the ID of any existing process group or session
- The child's parent *process ID* is the same as the parent's process ID
- The child does not inherit its parent's *memory locks* (`mlock`, `mlockall`)
- Process resource utilizations (`getrusage`) and CPU time counters (`times`⁷) are reset to zero in the child

⁶ These points are specified in POSIX.1

⁷ See the `times(3POSIX)` manual page

- The child's set of pending *signals* is initially empty (`sigpending`⁸)
- The child does not inherit *semaphore* adjustments from its parent (`semop`⁹)
- The child does not inherit process-associated *record locks* from its parent (`fcntl`), but it does inherit `fcntl` *open file description locks* and `flock` locks from its parent
- The child does not inherit timers from its parent (`setitimer`, `alarm`, `timer_create`¹⁰)
- The child does not inherit outstanding *asynchronous I/O operations* from its parent (`aio_read`, `aio_write`), nor does it inherit any asynchronous I/O contexts from its parent (`io_setup`)

The parent and child also differ with respect to the following Linux-specific process attributes

- The child does not inherit directory change notifications (`dnotify`) from its parent
- The `prctl PR_SET_PDEATHSIG`¹¹ setting is reset so that the child does not receive a signal when its parent terminates
- The default timer slack value is set to the parent's current timer slack value (`prctl PR_SET_TIMERSLACK`)
- *Memory mappings* that have been marked with the `madvise MADV_DONTFORK` flag are not inherited across a `fork`
- Memory in address ranges that have been marked with the `madvise MADV_WIPEONFORK` flag is zeroed in the child after a `fork` and the `MADV_WIPEONFORK` setting remains in place for those address ranges in the child
- The termination signal of the child is always `SIGCHLD`¹²
- The port access permission bits set by `ioperm`¹³ are not inherited by the child; the child must turn on any bits that it requires using `ioperm`

The following points are also true

- After a successful call, the entire *virtual address space* of the parent is replicated in the child
- After a `fork` in a *multithreaded program*, the child can safely call only *async-signal-safe* functions until such time as it calls `execve`
- The child inherits copies of the parent's set of *open file descriptors*. Each file descriptor in the child refers to the same *open file description* as the corresponding *file descriptor* in the

8 See the `sigpending(2)` manual page

9 See the `semop(2)` manual page

10 See the `setitimer(2)`, `alarm(2)`, `timer_create(2)` manual pages

11 See the `prctl(2)` manual page

12 See the `clone(2)` manual page

13 See the `ioperm(2)` manual page

parent. This means that the two file descriptors share open file *status flags*, *file offset*, and signal-driven I/O attributes

- The child inherits copies of the parent's set of *open message queue descriptors*. Each file descriptor in the child refers to the same *open message queue description*¹⁴ as the corresponding file descriptor in the parent. This means that the two file descriptors share the same flags
- The child inherits copies of the parent's set of *open directory streams*

After a fork all file descriptors that are open in the parent are duplicated in the child. The parent and the child share a *file table entry* for every *open descriptor*. A process that has three different files opened for stdin, stdout, and stderr has the arrangement shown by the following picture after a fork

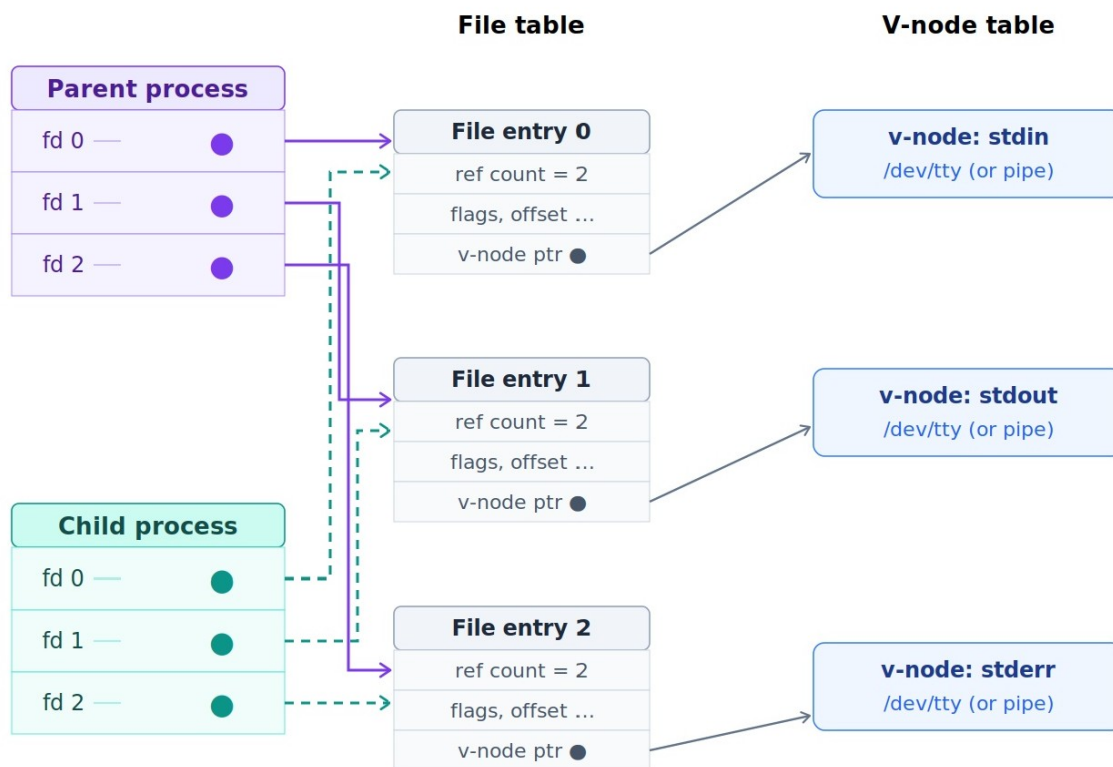


Fig 1. Sharing of open files after `fork`

If a descriptor was open before the `fork` and both parent and child write to the same descriptor, without any form of synchronization, such as having the parent wait for the child, their output will be intermixed. The *file offset* is shared, therefore if the parent waits for the child to finish, after the child has written data into the file the *file offset* is updated and the parent will write starting at that offset.

¹⁴ See the `mq_overview(7)` manual page

After `fork`, there is no guarantee whether the parent or the child will run first; the execution order depends on the kernel's scheduling decisions. If the parent and child must coordinate their actions, some form of *interprocess synchronization* or *communication* is required, such as `wait`, *signals*, or other IPC mechanisms.

In order to achieve synchronization between the processes we can use

- `wait/waitpid`
Parent-child synchronization mechanism that allows a parent process to block until a child terminates (or changes state) and to retrieve the child's exit status. Prevents zombie processes
- *Signals* (`SIGCHLD`, user-defined)
Asynchronous notifications sent by the kernel or processes to inform a process that an event has occurred (e.g., child termination)
- *Pipes*
Unidirectional byte-stream IPC mechanism that allows one process to write data and another to read it
- *POSIX semaphores*
Kernel-managed or user-space counters used to control access to shared resources or to enforce ordering between processes or threads
- *Shared memory* with synchronization primitives
Fastest IPC mechanism where multiple processes map the same memory region.

The maximum number of child processes a real user ID can create is defined by the `CHILD_MAX` constant. We can retrieve the value from the shell as follows

```
[linux@process] getconf CHILD_MAX
61953
```

Or in the code via `sysconf`

```
printf("%d", sysconf(_SC_CHILD_MAX));
```

One of the main uses of `fork` is to duplicate a process so that the parent and the child can execute different sections of code concurrently. This pattern is commonly used in network servers: the parent process waits for incoming client requests, and when a request arrives, it calls `fork` to create a child process that handles the request, while the parent immediately returns to waiting for the next request.

Another main use of `fork` is when a process wants to execute a different program. This pattern is common in *shells*: the shell calls `fork` to create a child process, and the child immediately calls an `exec` function to replace its *address space* with the new program, while the *parent shell* continues running.

It is possible to register *handlers* that are executed when `fork` is called by any thread in a process. To register one or more such handlers, a program can use the `pthread_atfork` function¹⁵. If we want to avoid any handler to be executed when `fork` is called we can use the `_Fork` function that is defined as follows

```
#include <unistd.h>

pid_t _Fork(void);
```

This `_Fork` function is equivalent to `fork`, except that *fork handlers* registered with `pthread_atfork` are not called

15 See the `pthread_atfork(3)` manual page